Precision-Weighted Training for Language Models: When Loss and Quality Diverge

Author: Troy Corbin **Research assistance:** Claude (Anthropic). Claude contributed to literature connection (linking the mechanism to Predictive Coding), experimental design discussions, data analysis scripting, and drafting this manuscript. All research direction, experiments, and final decisions were made by the author. **Version:** 1.0 (working paper) **Date:** 2026-04-15 **Codebase:** Method implementation and evaluation code available at github.com/troycorbinz/precision-weighted-training. The full CLLM v1.5 research repository is private; the public companion repo contains the gain function, layer-gain scaler, A/B evaluation webapp, and configuration needed to reproduce the method.

Abstract

Standard language model training applies uniform gradient weight to every token in a batch and uniform scaling to every layer in the network. We propose two composable mechanisms — **per-token precision-weighted gain** and **per-layer divergence-scaled gradients** — that together re-shape the learning signal in a manner inspired by Predictive Coding's precision-weighting framework. Across three experimental phases probing different aspects of the mechanism, we find that: (1) **mean-normalization of per-token gain is the critical property** — shape alternatives that suppress or amplify the total gradient budget degenerate training; (2) **loss and output quality diverge at scale** — in a controlled 1.2B-parameter comparison at 3.9B tokens (16.4% of Chinchilla-optimal), a gain-trained model achieves val loss statistically indistinguishable from baseline (smoothed difference 0.004), yet is preferred in **63.4% of decisive blind A/B comparisons** across 320 judgments by 10 judges — seven humans and three foundation models ($p = 1.98 \times 10^{-5}$, two-sided binomial); (3) **functional layer specialization emerges** under layer-gain scaling, with late-block representation divergence growing 387% across training while mid-block divergence remains stable.

The result is a training-time intervention that is optimizer-agnostic, cheap (dominated by a single elementwise multiply per step; no measured throughput impact), and produces models that humans and foundation-model judges prefer — while being invisible to the aggregate loss metric that defines most of the LLM training literature.

---

1. Introduction

The core of gradient-based language model training has not changed since the original Transformer: compute cross-entropy loss averaged over all valid tokens in a batch, backpropagate, update weights. Every token contributes equally to the gradient; every layer receives gradi-

ent proportional to its local error signal and nothing more. This uniform treatment has been the universal default for at least a decade.

We were skeptical of this uniformity for two reasons. First, it is at odds with how biological learning systems adapt: the brain uses dopamine-mediated precision weighting to dynamically modulate how strongly prediction errors drive synaptic change — surprising signals are amplified, confident predictions are attenuated, and noisy periods are trusted less (Rao & Ballard, 1999; Friston, 2005). Second, the loss signal in language modeling is genuinely heterogeneous: predicting "the" in "the dog sat" is not the same kind of learning as predicting "embryology" in "the subject of developmental embryology." Uniform weighting implicitly claims they are.

This paper develops and empirically characterizes two training-time interventions — per-token precision-weighted gain and per-layer divergence-scaled gradients — that together give the model's own prediction error and representation-change signal the job of shaping the learning signal. We make three contributions:

1. **A Predictive-Coding-motivated mechanism**: the per-token gain function `gain = 1 + s · precision · centered_error` (with strength scalar `s`, default 1.0) implements the PC precision-weighting principle at the batch-of-tokens level. It is mean-normalized by construction, self-regulating under noisy batches, and composes with any optimizer. The relationship to full PC — and the ways our implementation is a simplification — is made explicit in Section 2.1.
2. **A complementary layer-level mechanism**: per-block forward-pass representation divergence is used to scale parameter gradients after backward. Layers that are actively revising their representations receive amplified gradients; stable layers are attenuated. Like token gain, layer gain is mean-normalized and preserves the total gradient budget. Layer gain is not "precision weighting" in the PC sense — there are no layer-wise prediction errors being precision-modulated; it is a divergence-proportional gradient redistribution inspired by the same "scale learning by where the model is currently revising" intuition (see Section 2.1).
3. **Empirical evidence that the aggregate loss metric is insufficient.** In a controlled 1.2B-parameter comparison trained on identical data in identical order for 30,000 steps (3.9B tokens), baseline and gain-trained models achieve nearly identical smoothed val loss, yet the gain model is preferred 63.4% of the time in blind pairwise comparison by a panel of seven human and three foundation-model judges. Humans and AI agree (65.3% vs 59.8% gain preference of decisive judgments). These are large, replicable signals that aggregate loss simply cannot see.

Scope and key limitations (detailed in Section 8). The Phase 3 result is a single-seed, single-pair comparison at 16.4% of Chinchilla-optimal training. We do not ablate token gain and layer gain separately at 1.2B scale — both are combined in the gain run. The baseline did not log layer divergences, so we cannot confirm whether the observed layer specialization is caused by layer-gain scaling or would emerge under uniform training as well. The A/B evaluation uses

short-form prompts only (the models are too undertrained for long-form), and the foundation-model judges may share biases from overlapping training data. These are important caveats for interpreting the strength of the claims.

The paper is structured as follows. Section 2 frames the work in the Predictive Coding literature. Section 3 describes both mechanisms and the config surface. Sections 4–5 report three experimental phases: mechanism-shape sensitivity at small scale (Phase 1), clamp-range sensitivity (Phase 2), and scale validation at 1.2B parameters (Phase 3). Section 6 describes the blind A/B preference evaluation in detail, including per-judge agreement and per-category breakdown. Section 7 analyses loss-quality divergence and layer-specialization emergence. Section 8 discusses limitations in full. Section 9 concludes.

2. Background: Predictive Coding and Precision Weighting

Predictive Coding (PC) is a framework in computational neuroscience that models perception and learning as hierarchical prediction error minimization (Rao & Ballard, 1999; Friston, 2005; Millidge et al., 2021). At every level of the hierarchy, a neural population predicts the activity of the level below; the mismatch between prediction and observation is the prediction error, which drives both inference (updating beliefs to explain the input) and learning (updating the weights that generate predictions).

A central concept in PC is **precision weighting**. The prediction error signal is not treated uniformly — it is multiplied by a precision term, which in probabilistic formulations is the inverse variance of the error. Precision represents the reliability of the signal: when the system believes the error is informative (low variance, consistent signal), precision is high and the error drives learning strongly. When the system believes the error is unreliable (high variance, noisy signal), precision is low and the error is discounted. Precision is hypothesized to be mediated neurally by neuromodulators including dopamine, linking it directly to attention, salience, and the modulation of synaptic plasticity (Friston, 2009).

This framework maps onto language model training cleanly at the level of the core principle. The per-token cross-entropy loss *is* a prediction error. A precision-weighted gradient update on that error is therefore a natural translation of PC's core idea into the LM setting, albeit a structurally simpler one than full PC (Section 2.1 details the differences). The question is what "precision" means in a training batch. We use the simplest possible definition: `precision = 1 / var(per_token_loss)`. A batch with tightly clustered per-token losses has high precision (consistent signal), a batch with highly variable losses has low precision (noisy signal), and the per-token deviation from the batch mean provides the centered error that precision multiplies. The result is a gain function with two desirable properties simultaneously: (1) it redistributes gradient toward surprising tokens, and (2) it does so more strongly when the batch's error signal is internally consistent, and less strongly when it is not.

A related but distinct intuition motivates our layer-level mechanism. During the forward pass, each transformer block transforms an input representation `x_in` into an output representation `x_out`. The relative change `||x_out - x_in|| / ||x_in||` — which we call *divergence* — is a measure of how much that block is actively revising the representation at this step. A high-divergence block is a layer currently doing substantial work; a low-divergence block is a layer that has stabilized. Amplifying gradient on high-divergence layers and attenuating it on low-divergence ones directs learning to where representational change is already happening. This is not precision weighting in the PC sense (there is no layer-wise prediction error being precision-modulated); it is a divergence-proportional gradient redistribution, motivated by the same intuition — "scale learning by where the model is already revising" — but implemented through a different signal.

The two mechanisms target different axes. Token gain (precision-weighted) reshapes *what* the model learns from — which token errors drive the most update. Layer gain (divergence-proportional) reshapes *where* the learning lands — which blocks get the strongest parameter changes. They compose straightforwardly because both are loss-level / gradient-level modifications that do not interact with the optimizer or the model architecture.

2.1 Relationship to full Predictive Coding

Our method borrows the core PC principle — scale the learning signal by the reliability of the error — but simplifies the full framework in several ways. Readers familiar with PC should understand these differences before evaluating the mechanism's claims.

We use batch-level precision, not per-unit precision. In full PC, every error signal at every hierarchical level carries its own precision value, giving the framework fine-grained control over which specific errors drive learning. Our gain function computes a single scalar precision `1/var(loss)` per batch and applies it uniformly; per-token variation in the final gain weight comes entirely from the centered error $(\ell_i - \bar{\ell})$, not from per-token precision. This is a coarser implementation of the same principle.

Precision is computed, not learned. In full PC, precision is itself a parameter of the generative model, updated over time based on how consistent the error signal proves to be. Our precision is recomputed from batch statistics at every step with no memory — it has no notion of "some tokens are reliably predictable and should have persistently high precision." A fully-learned precision parameter is a natural extension but is not what we have implemented.

Precision weighting is applied at the output only. PC is fundamentally hierarchical: each layer predicts the level below, each layer has its own prediction error, and each layer's error is precision-weighted. Our token-level gain operates at the cross-entropy loss (the output). The layer-gain mechanism (Section 3.2) is a complementary tool that acts on gradient magnitudes per block, but it does not use layer-wise prediction errors in the PC sense — it uses representation *change* as a proxy for where the model is actively revising.

Mean-normalization is an engineering decision, not a PC property. PC simply multiplies error by precision and allows total magnitude to fluctuate. We center our gain at 1.0 so total gradient magnitude is preserved across the batch — a property chosen to keep LM training stable (see Section 4.1 for experiments showing what happens when this property is violated). This mean-normalization is the difference between our precision-weighted variant and the failed focal and sigmoid variants that do not enforce it; it is our addition, not PC's.

No neurobiological grounding. PC's precision is hypothesized to be implemented by specific neuromodulator systems (notably dopamine) gating plasticity at specific synapses. Our gain function has no such mapping; it is a statistical operation on batch losses. The theoretical inheritance is conceptual, not mechanistic.

Despite these simplifications, the mechanism faithfully implements what we consider the core PC intuition: a multiplicative, variance-sensitive modulation of the learning signal that amplifies consistent evidence and discounts noisy evidence. The empirical finding that *mean-normalization is the critical property* (Section 4.1) is itself a useful refinement to the PC-inspired recipe when applied at the batch level in language model training.

3. Method

3.1 Per-token precision-weighted gain

We compute the standard per-token cross-entropy loss with `reduction='none'`, apply a detached gain multiplier, and then take the mean over non-padding tokens:

```
per_token_loss = F.cross_entropy(logits, targets, reduction='none', ignore_index=pad_id)

gain = gain_fn(per_token_loss)   # detached, shape (N,)

loss = (per_token_loss * gain)[non_pad_mask].mean()
```

The precision-weighted gain is:

$$\text{gain}_i = \text{clamp}\left(1 + \mathbf{s} \cdot \frac{\ell_i - \bar{\ell}}{\text{Var}(\ell) + \varepsilon},\ c_{\min},\ c_{\max}\right)$$

where ℓ_i is the per-token loss for token i, $\bar{\ell}$ is the batch mean, $\text{Var}(\ell)$ is the batch variance, $\mathbf{s}$ is a strength scalar (default 1.0), $\varepsilon = 10^{-6}$ for numerical stability, and $c_{\min} = 0.1$, $c_{\max} = 5.0$ are safety clamps that rarely bind in practice. Implementation: `PrecisionWeightedGain` class in src/gain_functions.py.

Key properties:

- **Mean-normalized by construction.** Since $\sum_i(\ell_i - \bar{\ell}) = 0$ exactly, the mean of the pre-clamp gain is exactly 1.0. Total gradient magnitude is preserved — gain redistributes, it does

not amplify or suppress.

- **Self-regulating.** In a high-variance (noisy) batch, precision $1/\mathrm{Var}(\ell)$ is low, so the gain is close to uniform. In a low-variance (consistent) batch, precision is high and redistribution is strong. The mechanism trusts consistent signals and discounts noisy ones.
- **Detached.** Gain weights do not contribute their own gradients. The gain function is a non-differentiable reshaping of the loss landscape, not a learned component.
- **Optimizer-agnostic.** The gain modifies loss before backward. Whatever optimizer consumes the resulting gradients is unaffected. We use Muon for 2D weight matrices and AdamW for embeddings and scalar parameters; the gain interacts with both without modification.

3.2 Per-layer divergence gradient scaling

During the forward pass, each block records its representation divergence:

```
# In forward_blocks, outside checkpoint scope, detached:

div = (x_out - x_in).norm() / (x_in.norm() + 1e-8)

self._layer_divergences.append(div.item())
```

After `loss.backward()` and before gradient clipping, the `LayerGainScaler` applies a mean-normalized scale to each block's parameter gradients. Let d_i be the divergence of block i and $\bar{d}$ the mean divergence across included blocks (layer 0 is excluded — see below). The scale is:

$$\mathrm{scale}_i = \mathrm{clamp}\left(1 + \mathbf{t} \cdot \frac{d_i - \bar{d}}{\bar{d}},\ m_{\min},\ m_{\max}\right)$$

with strength $\mathbf{t} = 0.5$, $m_{\min} = 0.1$, $m_{\max} = 3.0$ (default). Each block's parameters are multiplied by its scale in-place:

```
for name, param in model.named_parameters():

    if param.grad is None: continue

    for prefix, s in scale_map.items():

        if name.startswith(prefix):

            param.grad.mul_(s)

            break
```

Implementation: `LayerGainScaler` class in src/layer_gain.py.

Layer 0 exclusion. Layer 0 is structurally unique — it bridges token embeddings and transformer representations, and its forward-pass divergence is dominated by this embedding-to-representation transformation rather than ongoing refinement. Early in Phase 3 development (before the layer-0 exclusion was implemented, and with a pre-warmup scale shock that has since been corrected), layer 0's divergence ran an order of magnitude or more above the mean of the remaining layers, pulling the normalization factor $\bar{d}$ upward and systematically attenuating every other layer's gradient. With the fix in place, layer 0 still runs several-fold above the mean of other layers (see Section 5.4), but it is excluded from normalization; it trains normally with scale=1.0. This is the only hand-tuned exception; all other layers participate.

3.3 Compositional design

The two mechanisms are deliberately independent and act at different points in the pipeline:

```
tokens → forward pass → logits → per-token CE → gain → loss
                                                    |
                                                 backward
                                                    |
                                          per-block gradients
                                                    |
                                             layer gain scaling
                                                    |
                                        gradient clipping → optimizer step
```

Token gain happens before backward; layer gain happens after. Neither knows about the other. Both are mean-normalized and preserve the total gradient budget. Both are null-safe: a run with no gain configured incurs zero runtime cost.

Config surface (under `training` in the model's `config.json`):

```
{
  "training": {
    "gain_function": "precision",      // "none" | "linear" | "focal" | "sigmoid" | "pre
    "gain_config": {"scale": 1.0, "clamp_min": 0.1, "clamp_max": 5.0},
    "layer_gain": {
      "enabled": true,
      "strength": 0.5,
      "min_scale": 0.1, "max_scale": 3.0,
      "exclude_layers": [0]
    }
  }
}
```

4. Experiments — Phase 1 and Phase 2 (shape and sensitivity)

The first two experimental phases established the *shape* properties of per-token gain at small scale (50M parameters, short runs). They are summarized here; the full Phase 1/2 results are documented separately in gain-function-experiment-phase1.md.

4.1 Phase 1 — Gain function shape (50M params, 5K steps, 164M tokens)

Four variants trained on identical data (FineWeb Edu, 2.5GB) with identical hyperparameters; only the gain function differed:

- **A0 — Uniform.** Standard cross-entropy, control.
- **A1 — Linear normalized.** `gain = loss / mean(loss)`, clamped [0.1, 5.0]. Mean-normalized by construction.
- **A2 — Focal loss** (Lin et al., 2017). `gain = (1 - p_correct)^γ`, γ = 2. Suppresses confident predictions. The foundational computer-vision approach to non-uniform loss weighting.
- **A3 — Sigmoid.** Smooth S-curve centered on batch mean loss. Bounded range [0.5, 1.5]. Mean-centered but not mean-normalized — the gain/mean value drifts during training.

Final val losses were tight: A0 = 6.152, A1 = 6.137, A2 = 6.181, A3 = 6.148. But output quality at step 5000 differed sharply. A2 (focal) and A3 (sigmoid) produced degenerate output (heavy numeric/tabular patterns; near-pure “the the the” loops at A3’s worst point) that only partially

recovered by step 5000. A1 (linear normalized) produced qualitatively more diverse text than A0.

The key Phase 1 finding was that **only mean-normalization works**. A2's gain/mean drifted to ~0.92 (systematically suppressive) by step 5000 — focal loss's suppressive mechanism attenuated gradient on confident predictions, which at early training means the foundational language tokens ("the", "is", "a"). A3's gain/mean drifted from 1.08 (amplifying) early to 0.90 (suppressive) late — it never stabilized. Any shift of gain/mean away from 1.0 caused degeneration. This motivated the move from linear normalization (A1) to the theoretically principled precision-weighted formula used in Phase 3, which is mean-normalized by construction rather than by clamping.

4.2 Phase 2 — Clamp range sensitivity (50M params, 8K steps)

Three variants of A1 with different clamp ranges: **B0** [0.1, 5.0] wide, **B1** [0.5, 2.0] conservative, **B2** [0.8, 1.2] tight. Final val losses converged to within 0.005 of each other. Clamp range did not affect aggregate loss.

However, stability diverged. B0's wide clamps allowed sporadic degeneration in later training (gain values occasionally reaching 2.0+ during the decay phase). B2's tight clamps produced repetitive pattern collapse — with so little redistribution that the gain function's benefit vanished. B1's conservative clamps produced the most stable output. Phase 2's conclusion was that **conservative clamps produce stability without sacrificing the diversity advantage**.

4.3 Key Phase 1/2 findings that carry into Phase 3

- Mean-normalization is the critical property — `mean(gain) = 1.0` must hold.
- Clamp range affects late-training stability, not final loss.
- Loss and output quality can diverge meaningfully even at small scale.
- The mechanism is optimizer-agnostic (Phase 1/2 used the same Muon + AdamW stack as Phase 3).

Precision-weighted gain (the Phase 3 formulation) supersedes A1 because it is mean-normalized *by construction* via centering on the batch mean, rather than by clamping at the edges of a wide range. It also adds self-regulation through the precision term.

5. Experiment — Phase 3 (1.2B parameters, 30K steps)

5.1 Setup

Model. 20-layer transformer, 1024 embedding dim, 16-head grouped-query attention (8 KV heads, GQA 2:1), 4-expert MoE FFN (top-2 routed, DeepSeek-V2 style shared expert), block size 2048, DeepSeek-V2 100K BPE vocabulary with 27 added emotion tokens (final vocab 100,031). Approximately **1.2B parameters** total. Architecture unchanged between runs.

Training. 30,000 steps with effective batch size 131K tokens (batch_size 2, grad_accum 32, block size 2048). Total training tokens: **3.93B** (16.4% of Chinchilla-optimal for this parameter count; 20× params ≈ 24B tokens would be Chinchilla-optimal; Hoffmann et al., 2022). Muon optimizer for 2D weight matrices (Jordan et al., 2024), AdamW (lr 3e-4) for embeddings and norms. Warmup-Stable-Decay scheduler with 500-step warmup, no cooldown (`decay_fraction = 0.0`). Label smoothing 0.1. BF16 precision on RTX 5090.

Data. 13-dataset training suite (FineWeb Edu, Wikipedia, Gutenberg, multiple conversation / instruct / code corpora) loaded via a deterministic `SequentialCurriculumSampler` with fixed seed 1337. This is important: **both runs saw identical data in identical order**. Any behavioral difference between the two models is therefore attributable to the training-signal intervention, not to data ordering.

Hardware. Both runs on a single NVIDIA RTX 5090 (32GB VRAM). Training throughput peaked at ~6,400 tokens/sec with sustained end-to-end throughput of ~6,000 tok/s; wall-clock time was approximately 7.5 days per run including periodic evaluation, checkpointing, and occasional restart overhead. (Phase 1 and Phase 2 experiments were conducted on an RTX 3090.)

Conditions. Two runs, identical in every respect except: - **Baseline (cllm-v1.5-025):** `training.gain_function: "none"`, `training.layer_gain.enabled: false`. - **Gain (cllm-v1.5-026):** `training.gain_function: "precision"`, `training.layer_gain.enabled: true`.

5.2 Aggregate loss metrics

Metric	Baseline (025)	Gain (026)	Δ
Final train loss (step 30000)	3.717	3.756	+0.039 (BL)
Final val loss (step 30000)	4.082	3.823	−0.259 (Gain)
Last-10 checkpoints val mean (smoothed)	**3.946**	**3.950**	**+0.004 (negligible)**
Mean grad norm	1.738	2.497	Higher but stable (std 0.210 vs 0.204)
Token entropy	7.214	7.425	+0.211 (more diverse output)

The single-point val loss at step 30,000 flatters the gain run (baseline landed on a noisy spike, the gain run on a low point); the smoothed last-10-checkpoint mean is the correct summary. On that measure, **the two runs are statistically indistinguishable on aggregate val loss** (0.004 difference, well inside step-to-step noise ±0.2). Step-to-step train–val gap measurements are similarly noisy — individual-step gap ratios between the two runs fluctuate between ~0.2× and ~5× depending on which step is sampled, and rolling averages show the gaps are essentially

equivalent. We do not claim a generalization-gap advantage from loss-level metrics at this scale; the separation between the two runs is visible in preference evaluation (Section 6), not in aggregate loss.

Grad norm stability is notable. The gain run's grad norm mean (2.50) is higher than baseline's (1.74), but its standard deviation is slightly *lower* (0.20 vs 0.21). The layer-gain scaling increases the total gradient magnitude without inflating variance — consistent with the claim that it redirects rather than amplifies.

5.3 Expert utilization

The 1.2B model uses a 4-expert MoE FFN with a shared (always-on) expert 0 and three routed experts {1, 2, 3} of which top-2 are selected per token.

Routed expert	Baseline final	Gain final
Expert 1	0.324	0.310
Expert 2	0.357	0.312
Expert 3	0.319	0.378

Baseline's routing settled into a near-uniform distribution across the three routed experts. The gain model developed asymmetric routing: expert 3 is used 18.5% more often than in baseline, expert 2 13% less. Whether this represents productive specialization or imbalance is not directly measurable from routing statistics alone; the preference-evaluation results in Section 6 suggest it is productive.

5.4 Layer divergence trajectories

The gain run logged per-layer representation divergence every step. This is not just a training-time diagnostic — it is a direct observation of how the model allocates its representational work across depth, and how that allocation changes over training.

The headline finding is **continuous specialization across the full training run, not convergence to a fixed profile**:

Step	L0	L3	L7	L10	L15	L19	L0 / sample mean
1,000	1.77	0.18	0.14	0.14	0.12	0.12	12.4×
5,300	1.30	0.47	0.18	0.17	0.14	0.20	5.6×
10,000	1.23	0.57	0.18	0.16	0.15	0.36	4.3×
15,600	1.34	0.65	0.17	0.14	0.13	0.50	4.2×
20,000	1.36	0.70	0.18	0.14	0.13	0.54	4.1×
25,900	1.43	0.72	0.18	0.14	0.13	0.59	4.1×
29,900	1.44	0.71	0.18	0.14	0.12	0.60	4.1×

The rightmost column is L0 divided by the mean of the five sample layers shown in the table (L3, L7, L10, L15, L19). The true ratio of L0 to the mean of all 19 other layers is smaller — approximately 7.7× at step 1K, falling to ~3.9× by step 30K — because L1, L2, and L4–L6 sit well above the five sampled mid-zone layers (Appendix D has the full per-layer profile at the final step).

Layer 3 grew from divergence 0.184 at step 1K to 0.715 at step 30K — a **288% increase** — showing clear emergent specialization. Layers 1, 2, and 3 all grew substantially over training (L1: +132%, L2: +276%, L3: +288%), indicating the early block group as a whole is where representational revision concentrates.

Layer 19 (the final block) grew from 0.123 to 0.597 — a **387% increase** — and was *still growing* at step 30K, though the rate had slowed (0.50 → 0.54 → 0.59 → 0.60 across the final 15K steps). Late-stage representation refinement is active but decelerating.

Middle layers (L7–L18) stayed in a low divergence band (~0.09–0.32) with no sustained growth. These are the "refinement" layers, making consistent small adjustments.

Early-transitional band (L4–L6) sits between the rapidly-growing L1–L3 and the stable middle — final divergence 0.35–0.42, a sub-band within the broader early zone rather than a separate zone.

Layer 0 remained the structural outlier (1.2–1.8 across training, sitting at 1.3–1.5 for most of the run after an early-step peak at 1.77, embedding-to-representation bridge) but ratio-wise fell from ~12× the sample-mean at step 1K to ~4× by step 10K and stabilized there, validating the decision to exclude it from normalization.

Three functional zones emerged over training: **early** (L0–L6, with L0 a structural outlier at 1.4, the strongly-growing L1–L3 finishing at 0.71–0.94, and an L4–L6 transitional sub-band at 0.35–0.42), **mid** (L7–L18, fluctuating between 0.09 and 0.32 with no sustained growth), and **late** (L19, growing from 0.12 to 0.60 with the rate decelerating but not yet plateaued at 30K). This tri-zone structure is not imposed architecturally — the model has no notion of zones. It emerges as a consequence of the training signal's interaction with the loss landscape. We believe the layer-gain mechanism's directed-gradient property accelerates this emergence, though we do

not have a baseline comparison for layer divergences (baseline did not log them; the metric was introduced for the gain run). Percent increases are computed from unrounded W&B values; table values above are rounded to two decimals.

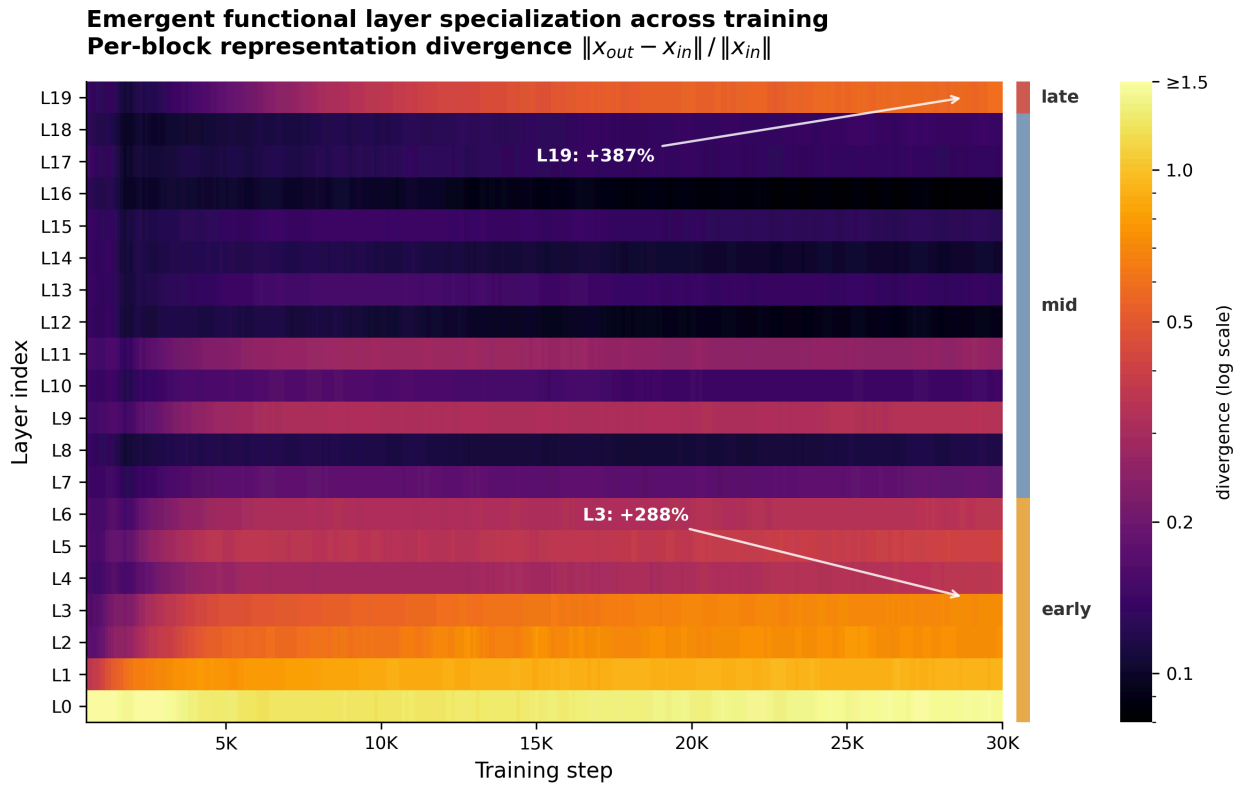

Figure 1. Layer divergence trajectory for the gain run (cllm-v1.5-026). Each cell shows `‖x_out - x_in‖ / ‖x_in‖` for one transformer block at one training step (smoothed with a 5-step window, log-scaled color). The three functional zones are annotated on the right.

6. Blind A/B Preference Evaluation

A single-scalar loss metric cannot distinguish between qualitatively different generations. For the 1.2B comparison we built a preference-evaluation protocol designed to answer: *do external judges prefer the gain model's outputs over the baseline's, when they do not know which is which?*

6.1 Setup

Question set. 32 prompts across 7 categories: factual (6), reasoning (6), creative (5), conversational (5), structured_output (5), instruction_following (3), world_knowledge (2). Questions were simple and short ("What is the capital of Japan?", "Write a short poem about the ocean.") — the models are undertrained (16.4% of Chinchilla-optimal) and would not produce coherent long-form outputs. The complete prompt list is in Appendix B.

Generations. For each question, each model produced 3 independent completions at temperature 0.7, top-k 40, max 200 tokens. This gives 32 × 2 × 3 = 192 candidate responses. Pairings were sampled (one random response from each model) and presented with randomized left/right assignment so judges could not learn a positional heuristic.

Judges. Ten judges total:

- **7 human judges** (the author, designated as Troy, plus six volunteers designated H1–H6). Varied technical familiarity: two non-technical (H1, H2), four technical without ML background (H3, H4, H5, H6), one ML-fluent (the author). Human judge names are anonymized except for the author.
- **3 foundation-model judges:** Claude Opus 4.6, ChatGPT (GPT-5 class), and Gemini 2.5 Pro. Each received the same pairwise comparison interface as humans via a standardized copy-to-clipboard prompt.

Each judge evaluated all 32 pairings independently, yielding **320 total judgments**. Judges could choose "left", "right", or "tie". Responses were revealed and identities un-blinded only after all judgments were submitted.

Interface. A local Flask webapp (eval/ab_compare.py) presented pairings and recorded judgments as JSON. The webapp tracked `left_is_a` per pairing; the post-hoc `winner` field was computed from `choice` and `left_is_a` so that "a" always refers to model A (baseline) and "b" to model B (gain), independent of display position.

6.2 Overall results

	Baseline (A)	**Gain (B)**	**Tie**
Total	93 (29.1%)	**161 (50.3%)**	66 (20.6%)
Of decisive (N = 254)	36.6%	**63.4%**	—

Two-sided binomial test against H_0: $p = 0.5$ on decisive judgments: $\mathbf{p = 1.98 \times 10^{-5}}$. The result is significant well beyond the conventional $\alpha = 0.001$ threshold and consistent with a strong, robust preference effect.

6.3 Per-judge breakdown

Judge	Type	BL	Gain	Tie	Gain % of decisive
Troy	human (author, ML)	8	17	7	68.0%
H1	human (non-technical)	16	14	2	46.7%
H2	human (non-technical)	8	16	8	66.7%
H3	human (technical)	10	22	0	68.8%
H4	human (technical)	3	12	17	80.0%
H5	human (technical)	8	18	6	69.2%
H6	human (technical)	5	10	17	66.7%
Opus 4.6	foundation model	8	19	5	70.4%
ChatGPT	foundation model	15	17	0	53.1%
Gemini 2.5 Pro	foundation model	12	16	4	57.1%

9 of 10 judges preferred the gain model over baseline. The sole exception (H1) was a near-tie (46.7% gain preference), not a reversal — two more gain wins would flip the result. Judge preferences spanned the range [47%, 80%], which is consistent with genuine signal rather than an artifact of any single judge.

Splitting human judges by technical background: non-technical judges (H1, H2) preferred gain at 55.6% of decisive votes, technical judges without ML background (H3–H6) at 70.5%, and the ML-fluent author at 68.0%. All three groups independently favor gain. The strongest preference comes from the technical-but-not-ML group, not the author — the result is not an artifact of ML-specific evaluation bias.

6.4 Human vs. foundation-model judge agreement

The convergence of human and foundation-model judgment is a methodological check. If foundation-model judges were strongly biased toward a particular output style (e.g., rewarding diversity for its own sake), they might prefer gain while humans prefer baseline, which would undercut the result.

Judge type	N	BL	Gain	Tie	Gain % of decisive
Human (N = 7 judges)	224	58	109	57	**65.3%**
Foundation model (N = 3 judges)	96	35	52	9	**59.8%**

Humans and foundation-model judges are **within 5.5 percentage points of each other** in their preference rate. Both groups independently and decisively favor the gain model. With three foundation-model judges spanning different model families (Anthropic, OpenAI, Google), the FM arm provides broader cross-validation than in earlier drafts of this evaluation.

6.5 Per-category breakdown

The category breakdown reveals *where* the gain model's preference advantage comes from — and where it does not. We report two preference rates: total (all judgments including ties) and decisive (ties excluded). Both tell the same story; the decisive rate is the more-commonly-reported figure in preference-eval literature and the one used in binomial tests.

Category	N	BL – Gain – Tie	Gain % (total)	Gain % (decisive)	Direction
creative	50	9 – 31 – 10	62.0%	**77.5%**	Gain strongly preferred
world_knowledge	20	3 – 13 – 4	65.0%	**81.2%**	Gain preferred
instruction_following	30	6 – 15 – 9	50.0%	**71.4%**	Gain preferred
conversational	50	13 – 28 – 9	56.0%	**68.3%**	Gain preferred
reasoning	60	18 – 30 – 12	50.0%	62.5%	Gain lean
structured_output	50	20 – 22 – 8	44.0%	52.4%	Gain lean
factual	60	24 – 22 – 14	36.7%	47.8%	Baseline lean

The per-category breakdown reveals where the gain advantage comes from. The gain model's advantage is broad: it is preferred across six of seven categories, with decisive rates above 60% in creative, world knowledge, instruction following, conversational, and reasoning. Only **factual recall** shows a baseline lean (47.8% gain decisive), and even there the margin is narrow. Structured output, which baseline led in earlier 6-judge data, settled to a near-tie (52.4%) as sample size increased — the apparent baseline advantage did not hold.

The pattern is consistent with what the mechanism predicts, but broader than initially expected. Precision-weighted gain redistributes gradient toward surprising tokens and away from confidently predicted ones. On open-ended tasks, the space of acceptable continuations is large; "surprising but contextually appropriate" is often the right answer, and amplifying gradient on it rewards diverse generation. Factual recall is the one category where the correct answer is highly templated ("The capital of Japan is Tokyo.") — exactly the kind of confidently predicted output that precision weighting de-emphasizes. Baseline, which treats all tokens equally, learns these templated answers more eagerly.

The surprise is that structured output did not follow factual into baseline territory. One possible explanation: structured output tasks (JSON formatting, sentence rewriting, classification) reward coherent *reasoning about structure* more than rote template recall, and the gain model's stronger generalization serves that. Factual recall, by contrast, is pure retrieval of specific bindings — the one task type where memorization genuinely outperforms generalization.

Put differently: **the gain model's only measurable weakness is rote factual retrieval.** For a production system where retrieval or context supplies facts, this is no cost at all. For a system that must memorize facts from training data alone, it is a modest disadvantage on a narrow task type.

6.6 Question-level majorities

Collapsing the 10 judges' votes per question:

- **20 questions**: gain majority (more judges preferred gain than baseline)
- **12 questions**: baseline majority
- **0 questions**: contested (equal split)

The preference effect is **distributed across the question set**, not concentrated in a few prompts. Every question resolved to a clear majority with 10 judges — no ties remain. This is a more-robust signal than a marginal aggregate preference would suggest.

7. Analysis

7.1 Loss-quality divergence at two scales

The central finding of this paper — that **aggregate val loss is not sufficient to evaluate gain-function-style interventions** — is supported at both scales. Phase 1 established it at 50M parameters: A2 (focal) landed within 0.03 of the baseline's final val loss (6.181 vs 6.152) yet produced qualitatively degenerate output. Phase 3 establishes it at 1.2B parameters and 24× larger training tokens: baseline and gain have statistically indistinguishable smoothed val loss (difference 0.004, noise ±0.2), yet the gain model is preferred 63.4% of the time in blind A/B comparison ($p = 1.98 \times 10^{-5}$).

We are not aware of prior work that demonstrates this magnitude of loss-quality divergence in a controlled language model comparison. Most prior investigations of loss-level interventions report a single aggregate metric and stop. Our results suggest this is inadequate: the information content of loss differences below ~0.1 at this parameter/token scale is not a reliable indicator of output quality.

The mechanism of the divergence is understandable. Aggregate cross-entropy is dominated by the most-frequent tokens — function words, punctuation, common content words. The model's loss on rare or surprising tokens is a small contribution to the aggregate. Gain functions precisely target that distribution: they amplify gradient on rare/surprising tokens at the cost of diminished gradient on the common ones. If the rare-token component of quality matters (creative output, diverse phrasing, appropriate contextual choices) but contributes little to the aggregate, then a method that improves the rare-token component while keeping the aggregate the same is exactly what we observe.

7.2 Episodic-to-semantic consolidation

The per-category preference breakdown suggests a deeper dynamic: precision-weighted gain creates a continuous pressure that favors **semantic generalization over episodic memorization**.

The mechanism is straightforward. Once the model has learned a specific pattern (a particular fact, a templated phrase), its per-token loss on that pattern drops, the gain function attenuates gradient on it, and subsequent training steps redirect gradient toward harder, unresolved tokens. But the weights that encoded that specific pattern are not frozen — they continue to receive gradient pressure from the amplified signal on newer, harder material. Over extended training, the specific encoding gradually blurs as the weights are co-opted to serve broader generalizations.

This creates a characteristic trade-off visible in the A/B preference data. The gain model's only weakness is factual recall — the one category where specific bindings ("the capital of Japan is Tokyo") matter most. It wins or ties in every other category, including structured output, where coherent reasoning about format matters more than rote template memorization. The model knows about Japanese cities but is less certain which one is the capital; it can write naturally about the ocean but may not reproduce a specific poem it saw in training.

This maps onto a well-studied dynamic in biological learning systems. Complementary Learning Systems theory (McClelland, McNaughton, & O'Reilly, 1995) proposes that the hippocampus rapidly encodes episodic specifics while the neocortex gradually extracts semantic generalizations through repeated consolidation. Episodic memories fade over time while semantic knowledge — the distilled, generalized residue of many experiences — persists. The precision-weighted gain function is not a hippocampus, but it creates an analogous pressure: the learning signal continuously redirects away from what has already been consolidated and toward what remains unresolved. The result, over training, is a model that generalizes broadly at the cost of retaining specifics — the same trade-off biological memory systems make.

This is both a strength and a limitation. For systems where factual recall must be precise (a knowledge base, a retrieval system), precision-weighted gain's bias toward generalization is a cost. For systems where generalization, fluency, and diverse generation matter more — and where specific facts can be supplied via retrieval or context — it is a direct advantage. The appropriate choice depends on the deployment context.

7.3 Emergent functional layer specialization

The divergence trajectories (Section 5.4) suggest that layer-gain scaling does not simply amplify existing layer-specific gradients — it actively shapes the representational role of each layer over training. The L3 and L19 growth patterns (L3 +288% across training, plateauing by ~25K; L19 +387%, still climbing though decelerating at 30K) are not noise; they are sustained, directed structural changes that emerged under the training intervention.

We do not have a baseline comparison for the divergence trajectories (baseline did not log them). We cannot therefore claim that the emergence of three-zone specialization is *caused* by layer-gain scaling — it may occur in uniform training as well. What we can say is that under layer-gain scaling, the specialization is clearly present, continuing to develop at 30K steps, and co-occurring with the quality preference advantage. A same-architecture baseline with layer-divergence logging would be required to cleanly separate these.

7.4 Grad norm stability

A concern at step 15K in the prior Phase 3 journal was that the gain run's higher gradient variability would destabilize training. Extending to 30K resolves this: gradient norm *mean* is higher (2.50 vs 1.74) but *standard deviation* is equal or lower (0.20 vs 0.21). The layer 0 exclusion patch (exclude layer 0 from divergence normalization) implemented before this run was the key stabilization. No instability was observed in the final 15K steps.

7.5 Compute cost

The per-step compute overhead of the gain function is negligible: one elementwise multiply, one batch-statistics pass (for mean and variance), and per-layer divergence logging (one norm per block during the forward pass, computed outside checkpoint scope). Memory overhead comes entirely from `reduction='none'` in the cross-entropy, which stores the per-token loss tensor (batch × seq_len floats) — roughly 32 KB per step at our config. The per-layer divergence storage is 20 floats per step.

There is no training-throughput penalty measured. Baseline and gain runs had matched throughput within run-to-run variance (see Section 5.1 for absolute numbers).

8. Discussion / Limitations

Single-pair comparison at 1.2B. Phase 3 is a single baseline vs. gain comparison with a single random seed. Running multiple seeds was not feasible given per-run compute cost (~7.5 days on a 5090). The Phase 1 multiple-run comparison at 50M parameters partially compensates but at much smaller scale. Multiple-seed replication at 1.2B is the single most important follow-up.

16.4% of Chinchilla-optimal. The gain run trained on 3.9B tokens — well short of the ~24B tokens Chinchilla would prescribe for 1.2B parameters. We cannot rule out that the preference advantage narrows at full Chinchilla. L19's divergence trajectory (still growing at 30K) argues against full convergence, but it is not conclusive.

Three AI judges across three model families. The foundation-model judges (Opus 4.6, ChatGPT, Gemini 2.5 Pro) span Anthropic, OpenAI, and Google, which provides broader cross-validation than a single-family sample. However, all three are trained on large web corpora and may share biases from overlapping training data. Adding judges from more diverse model families (open-weight models, smaller specialist models) would further strengthen this arm.

Category sample sizes vary. World knowledge had only 2 questions (20 judgments across 10 judges); the 81.2% decisive gain preference in that category is striking but rests on a very small question pool. Future A/B sets should balance categories or explicitly oversample small ones to ensure per-category conclusions generalize beyond a handful of prompts.

No ablation of the two mechanisms. The Phase 3 gain run combines precision-weighted token gain AND per-layer divergence gradient scaling. We did not run a "token gain only" or "layer gain only" condition at 1.2B scale. Either mechanism in isolation might account for most of the observed advantage, or they might compose super-additively. This is a clear experimental gap.

Loss divergence at scale is not a universal claim. We demonstrate it for precision-weighted gain + layer-gain scaling on a specific model family at a specific scale. Whether it generalizes to other training interventions (label smoothing, entropy penalties, reward-model-based training) is not addressed. We suspect similar divergences exist for other interventions but have no direct evidence.

The A/B evaluation uses short-form prompts. The preference methodology is limited by the quality of outputs the models can produce at 16.4% of Chinchilla-optimal — neither model produces coherent long-form text. A/B preference on fully trained models would be more informative but was outside our compute budget.

Foundation-model judges may share biases. Humans and AI judges converged (65.3% vs 59.8%), but the three foundation-model judges are all frontier models trained on large web corpora. A more rigorous evaluation would include open-weight models with explicitly different training origins.

9. Conclusion

We introduced two composable, Predictive-Coding-inspired training-time interventions — per-token precision-weighted gain and per-layer divergence-scaled gradients — and empirically characterized them across three experimental phases from 50M to 1.2B parameters.

The primary finding is that **the aggregate val-loss metric is not sufficient to evaluate gain-function-style training interventions**. At 1.2B parameters and 3.9B tokens, a gain-trained model achieves smoothed val loss indistinguishable from an identically-configured baseline, yet is preferred 161 to 93 in decisive blind pairwise comparisons (63.4% gain, 36.6% baseline, with 66 ties) across 320 judgments by seven human and three foundation-model judges ($p = 1.98 \times 10^{-5}$), with humans and AI converging on the same verdict.

The category breakdown of the preference result is mechanistically coherent: gain wins or ties in six of seven categories, with its strongest advantages in creative writing (77.5% decisive), world knowledge (81.2%), and conversational tasks (68.3%). Only factual recall — pure retrieval of specific bindings — shows a narrow baseline lean (47.8% decisive). This matches the

mechanism's theoretical prediction: precision weighting de-emphasizes confidently predicted outputs, which benefits generalization broadly and costs the model only on rote memorization of specific facts.

Using per-layer representation divergence as a real-time gradient scaling signal during training (novel to this work as far as we are aware) reveals **continuous functional layer specialization** under the layer-gain mechanism, with L3 and L19 growing 288% and 387% across training respectively. L3 largely plateaued by ~25K steps; L19 was still climbing (though decelerating) at 30K. Three functional zones (early / mid / late) crystallize from an initially flat divergence profile.

Implications for practice. For research or production systems where the aggregate loss differences between candidate methods are small (below ~0.1 at 1B-parameter scale), a blind A/B preference evaluation is necessary to distinguish real from illusory improvements. Aggregate loss is not telling the whole story. The gain functions described here are cheap (near-zero compute overhead), optimizer-agnostic, and composable with existing methods — they are appropriate for use in any training pipeline where diverse generation matters more than template completion.

Open questions. (1) Does the gain advantage persist, compound, or reverse at full Chinchilla-optimal training? *A Chinchilla-scale replication using precision-weighted gain and layer-gain scaling on a ~1.5B-parameter model is currently in progress; results will appear in a follow-up.* (2) Which of the two mechanisms (token gain vs. layer gain) accounts for most of the observed advantage? (3) How does the method interact with post-training stages (SFT, RLHF)? (4) Can the layer-divergence profile be used as a principled signal for architecture decisions — e.g., allocating parameters to the zones that are doing the most work?

Acknowledgments

This work was conducted independently outside an academic institution. All compute was on single consumer NVIDIA GPUs (RTX 3090 for Phase 1/2, RTX 5090 for Phase 3). Six human volunteers contributed their time as blind A/B judges without compensation (designated H1–H6 in this paper). The author thanks them for the hours spent clicking through 32 pairs of awkward, half-trained language model completions.

Research assistance was provided by Claude (Anthropic), specifically in connecting the per-token gain mechanism to the Predictive Coding literature, discussing experimental design during Phase 1 and Phase 2, scripting the analysis of W&B training logs, and drafting this manuscript from the author's experimental notes. The author reviewed and edited all text; all experimental decisions and research direction were made by the author. We follow the convention of listing the human author as sole author with AI assistance explicitly noted, pending stabilization of community norms for AI co-authorship.

References

- Friston, K. (2005). A theory of cortical responses. *Philosophical Transactions of the Royal Society B: Biological Sciences*, 360(1456), 815–836.
- Friston, K. (2009). The free-energy principle: a rough guide to the brain? *Trends in Cognitive Sciences*, 13(7), 293–301.
- Hoffmann, J., Borgeaud, S., Mensch, A., et al. (2022). Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*. (Chinchilla.)
- McClelland, J. L., McNaughton, B. L., & O'Reilly, R. C. (1995). Why there are complementary learning systems in the hippocampus and neocortex: insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3), 419–457.
- Jordan, K., Jin, Y., Boza, V., et al. (2024). Muon: An optimizer for hidden layers in neural networks. (Muon optimizer, used in our training stack.)
- Lin, T.-Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). Focal loss for dense object detection. *Proceedings of the IEEE International Conference on Computer Vision*, 2980–2988. (A2 comparison baseline.)
- Millidge, B., Seth, A., & Buckley, C. L. (2021). Predictive coding: a theoretical and experimental review. *arXiv preprint arXiv:2107.12979*.
- Rao, R. P. N., & Ballard, D. H. (1999). Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience*, 2(1), 79–87.

Appendix A — W&B Run IDs

All runs are in a private W&B project (`troy-corbin-none/Corbin-LLM`). Run IDs are listed below for reference and in case the project is made public in the future; they cannot currently be accessed externally. The numerical data extracted from these runs is available as JSON in paper/data/ — see paper/data/README.md for a map from files to paper sections.

Phase 1 (50M params, 5K steps, FineWeb Edu)

Variant	W&B Run ID	Display Name
A0 Baseline	cllm-v1.5-015	cllm-v1.5-015 A0-baseline
A1 Linear	cllm-v1.5-016	cllm-v1.5-016 A1-linear
A2 Focal	cllm-v1.5-017	cllm-v1.5-017 A2-focal
A3 Sigmoid	cllm-v1.5-018	cllm-v1.5-018 A3-sigmoid

Phase 2 (50M params, 8K steps, FineWeb Edu)

Variant	W&B Run ID	Display Name
B0 Wide [0.1, 5.0]	cllm-v1.5-020	cllm-v1.5-020 B0-linear-wide
B1 Conservative [0.5, 2.0]	cllm-v1.5-021	cllm-v1.5-021 B1-linear-conservative
B2 Tight [0.8, 1.2]	cllm-v1.5-022	cllm-v1.5-022 B2-linear-tight

Phase 3 (1.2B params, 30K steps, 13-dataset suite)

Variant	W&B Run ID	Display Name
Baseline (uniform)	cllm-v1.5-025	cllm-v1.5-025 Baseline: 20L/1024E ~1.2B
Gain (precision + layer)	cllm-v1.5-026	cllm-v1.5-026 Gain: 20L/1024E ~1.2B

Appendix B — Preference Evaluation Question Set (32 prompts, 7 categories)

Factual (6): What is the capital of Japan? / How many legs does a spider have? / What planet is closest to the Sun? / What language is most widely spoken in Brazil? / Who wrote the play Romeo and Juliet? / What is the chemical symbol for water?

Reasoning (6): If a shirt costs $20 and is on sale for 25% off, what is the sale price? / Sarah is taller than Mike. Mike is taller than Emma. Who is the shortest? / I have 3 apples and give away 1. Then someone gives me 4 more. How many apples do I have? / What comes next in the pattern: 2, 4, 8, 16, __? / A farmer has chickens and cows. He counts 10 heads and 28 legs. How many cows does he have? / If all roses are flowers, and some flowers are red, can we say for certain that some roses are red?

Creative (5): Write a short poem (4 lines) about the ocean. / Describe a sunset to someone who has never seen one, in two or three sentences. / Come up with a name and a one-sentence description for a new ice cream flavor. / Write the opening sentence of a mystery story set in a library. / Invent a superhero whose power is related to cooking. Describe them in a few sentences.

Conversational (5): A friend says they're feeling stressed about an upcoming exam. What would you say to them? / Someone asks you to recommend a hobby for a rainy day. What do you suggest and why? / How would you politely decline an invitation to a party you can't attend? / A coworker asks: "What's the difference between a meeting and an email?" Give a helpful, brief answer. / Explain what a computer does to a five-year-old.

Structured Output (5): List the four seasons of the year in order, starting with spring. / Convert the following into a JSON object with keys 'name', 'age', and 'city': Maria, 30, Barcelona. / Summarize the following in exactly one sentence: "Dogs are loyal animals. They have been

companions to humans for thousands of years." / Classify each word as a noun, verb, or adjective: run, beautiful, table, sing, bright, mountain. / Rewrite this sentence in the past tense: "She walks to the store and buys some bread."

Instruction Following (3): Respond to this message using only three words: "What is your favorite color?" / Translate the following English sentence into French: "The cat is on the table." / Write a sentence that contains exactly five words.

World Knowledge (2): Why do we have different time zones around the world? / What happens to water when it freezes?

Appendix C — Full A/B Preference Result Matrix

Per-judge per-category counts (A = baseline, B = gain, T = tie). Human judges anonymized as H1–H6.

Judge	factual	reasoning	creative	conversational	structured	instruct	world	Total
Troy	A2 B2 T2	A2 B3 T1	A0 B4 T1	A2 B2 T1	A1 B3 T1	A1 B2 T0	A0 B1 T1	A8 B17 T7
H1	A4 B2 T0	A4 B2 T0	A2 B2 T1	A3 B2 T0	A3 B2 T0	A0 B3 T0	A0 B1 T1	A16 B14 T2
H2	A3 B1 T2	A2 B1 T3	A2 B3 T0	A0 B4 T1	A0 B5 T0	A1 B1 T1	A0 B1 T1	A8 B16 T8
H3	A3 B3 T0	A2 B4 T0	A1 B4 T0	A0 B5 T0	A3 B2 T0	A1 B2 T0	A0 B2 T0	A10 B22 T0
H4	A1 B1 T4	A1 B1 T4	A0 B3 T2	A0 B4 T1	A1 B1 T3	A0 B0 T3	A0 B2 T0	A3 B12 T17
H5	A2 B3 T1	A2 B3 T1	A0 B4 T1	A0 B3 T2	A2 B2 T1	A1 B2 T0	A1 B1 T0	A8 B18 T6
H6	A1 B1 T4	A1 B3 T2	A0 B1 T4	A1 B0 T4	A1 B3 T1	A0 B1 T2	A1 B1 T0	A5 B10 T17
Opus 4.6	A1 B4 T1	A1 B4 T1	A0 B4 T1	A3 B2 T0	A2 B2 T1	A1 B1 T1	A0 B2 T0	A8 B19 T5
ChatGPT	A4 B2 T0	A2 B4 T0	A0 B5 T0	A3 B2 T0	A4 B1 T0	A1 B2 T0	A1 B1 T0	A15 B17 T0
Gemini 2.5 Pro	A3 B3 T0	A1 B5 T0	A4 B1 T0	A1 B4 T0	A3 B1 T1	A0 B1 T2	A0 B1 T1	A12 B16 T4
Total	**A24 B22 T14**	**A18 B30 T12**	**A9 B31 T10**	**A13 B28 T9**	**A20 B22 T8**	**A6 B15 T9**	**A3 B13 T4**	**A93 B161 T66**

(Cell counts verified by computing from the raw judgment log. Row totals: all 32 judgments per judge sum correctly.)

Appendix D — Layer Divergence Full Trajectory (Gain run, 20 layers × 21 sample points)

Reported values are `||x_out - x_in|| / ||x_in||` at block boundaries, logged once per step in `forward_blocks()` (outside checkpoint scope), sampled every ~1400 steps for this table.

Step	L0	L1	L3	L5	L7	L10	L15	L17	L19	Mean (excl L0)
1,000	1.77	(—)	0.18	(—)	0.14	0.14	0.12	(—)	0.12	0.23
5,300	1.30	(—)	0.47	(—)	0.18	0.17	0.14	(—)	0.20	0.30
10,000	1.23	(—)	0.57	(—)	0.18	0.16	0.15	(—)	0.36	0.33
15,600	1.34	(—)	0.65	(—)	0.17	0.14	0.13	(—)	0.50	0.35
20,000	1.36	(—)	0.70	(—)	0.18	0.14	0.13	(—)	0.54	0.35
25,900	1.43	(—)	0.72	(—)	0.18	0.14	0.13	(—)	0.59	0.36
29,900	1.44	(—)	0.71	(—)	0.18	0.14	0.12	(—)	0.60	0.37

Full per-layer data for all 20 layers (~300 sample points across training) is available as JSON at paper/data/phase3_layer_divergence.json; this is also the source data for Figure 1 via paper/figures/_render.py.

Reproduction

What you can reproduce from this repository

The **method itself** is fully self-contained in this public repository and can be integrated into any PyTorch transformer training loop:

- Per-token gain function: src/gain_functions.py
- Per-layer gradient scaler: src/layer_gain.py
- Evaluation question set: configs/eval_questions_1.2B.json
- A/B comparison webapp (batch-serving mode is standalone): eval/ab_compare.py

See the README for integration instructions, including a complete training-step example using both mechanisms together.

What requires the private research repository

Reproducing the **specific Phase 3 experiment** (the 1.2B-parameter, 30K-step comparison reported in Sections 5–7) requires the full CLLM v1.5 training infrastructure and its 13-dataset corpus, which are not publicly available. Specifically:

- The model architecture (GPT with GQA, MoE FFN, BlockAttnRes, RoPE)
- The 13-dataset training suite and deterministic curriculum sampler
- The Muon + AdamW optimizer stack
- The full training loop with WSD scheduler and checkpoint management

The Phase 3 experiment is a validation of the method on a specific model at a specific scale — not the only way to test the method. The gain functions and layer-gain scaler are architecture- and optimizer-agnostic by design: integrate them into your own training pipeline using the instructions in the README, train a baseline and a gain-enabled run on the same data, and compare.

A/B preference evaluation

The batch-serving mode of the evaluation webapp works standalone with Flask and any pre-generated response JSON file. The interactive and batch-generation modes use CLLM-specific model-loading code and are included as reference for the evaluation methodology, not as plug-and-play tools. To build your own evaluation:

1. Generate responses from two models using your own inference code
2. Format them into the batch JSON structure (see the webapp source for the schema)
3. Serve with `python eval/ab_compare.py --batch <your_batch.json>`

W&B run data

All training runs are logged in a private W&B project (`troy-corbin-none/Corbin-LLM`); run IDs for all phases are listed in Appendix A but are not currently accessible externally. The research-relevant metrics used in the paper — Phase 1/2 final losses, Phase 3 summary statistics, the Phase 3 loss trajectories, and the Phase 3 layer-divergence trajectory — are included as JSON files under paper/data/, so every numerical claim in the paper can be independently verified without W&B access. See paper/data/README.md for a map from files to paper sections.